

important fact to remember while learning Waf is 'Waf commands map to Python functions'. In simple terms, for each Waf command there should be a Python function defined in the Waf script *wscript*. So now we can use the four Waf commands—*options*, *configure*, *build* and *test*, which are available with the Waf script, and all of them need a Python function to define their actions. Each of these four functions has a parameter defined for them, which can be further used by other functions. In our case, the variables used to denote these parameters are *opt*, *conf*, *bld* and *tst*, respectively.

The first Python function from our Waf script *wscript* invokes a command called *configure*. The *configure* command is used to check if the requirements for working on a project are met and if it is possible to store the executable file and other associated files in your destination directory. The parameter of the *configure* command is then stored for use by other commands, such as the *build* command. The function *options* defines a command named *options* which is called once, before any other command is executed in *wscript*. In our case, the function *options* with the parameter *compiler_c* is used to load the C compiler in a platform-independent manner. The term 'platform-independent' means that when using Waf, it doesn't matter whether you have gcc, LLVM or some other compiler to compile the given C program. Instead, Waf will simply identify a compiler capable of compiling C programs on your system or show an error message if it fails to find a suitable C compiler. Other extensions like *compiler_cxx*, *compiler_fc* and *compiler_d* are provided to support the compilation of projects developed using C++, FORTRAN and D, respectively.

The most important function in *wscript* is called *build*, which can be executed with the command *waf* or *waf build*. In our *wscript*, the second and third lines of code in the *build* function create object files for the source files *data.c* and *show.c*. The first line of code in the *build* function generates the final executable file *hello* by using the file *main.c* and the object files generated from the files *data.c* and *show.c*. So, in order to build our sample project, we only need the three functions—*options*, *configure*, and *build*. You might then wonder what the purpose of the function *test* is, in the given *wscript*.

User defined Waf commands

Earlier, we learned the rule 'Waf commands map to Python functions'. So if you want a user defined Waf command, all you need to do is define a Python function in *wscript*. Execute the command *waf test* on the terminal and see the output. I am sure you will be able to guess the output.

Multiple Waf scripts for a project

For the simple project we are discussing, we only need

a single *wscript*. It is possible to have more than one *wscript* for even a single project. Consider the *wscript* shown below in the current working directory of our project:

```
def configure(conf):
    print("Waf File Configured!!!")

def test(tst):
    print("This is the main wscript")
    tst.recursive('src')
```

The *wscript* shown below is in the directory *src* inside the current working directory.

```
def test(tst):
    print("This is the wscript from the directory
src")
```

The commands *waf configure* and *waf* executed from the current working directory will give you the output shown in Figure 3. The line of code *tst.recursive('src')* calls the *wscript* inside the directory *src* along with the *wscript* from the main directory of the project. Now execute the command *cd src* and *waf*. The output of these commands (shown in Figure 3) might seem a bit surprising because the execution of the *wscript* inside the current working directory and the *wscript* inside the directory *src* are both showing the same output.

Even if you are calling Waf from a directory containing a *wscript*, the *wscript* executed is the one that is configured last in one of the parent directories. So if you want only the *wscript* from the directory *src* to take charge of affairs, you need to add a *configure* function in it. This example also tells you something about the default destination of Waf. If you haven't provided the command *out = '.'* or say *out = './dest'*, a default directory called *build* is generated by Waf to store the executable and other associated files.

A few final words about Waf

So we have learned about the installation and use of Waf with a simple C program. But this example and article just cover the bare minimum essential for a novice to

```
[root@depu waf]# waf configure
Setting top to : /root/Desktop/waf1/waf
Setting out to : /root/Desktop/waf1/waf/build

Main Waf script Configured!!!
'configure' finished successfully (0.197s)
[root@depu waf]# waf test
This is the main wscript
This is the wscript from the directory src
'test' finished successfully (0.001s)
[root@depu waf]# cd src
[root@depu src]# waf test
invalid lock file in /root/Desktop/waf1/waf/src
This is the main wscript
This is the wscript from the directory src
'test' finished successfully (0.001s)
```

Figure 3: Execution of multiple Waf scripts